

RentPoint

Automated Web-Based Property and Rental Management System for Nsukka Urban

Comprehensive Technical & Architectural Master Document

This document provides an exhaustive, end-to-end breakdown of the RentPoint system from initial scratch development to full production deployment. It covers all frameworks, libraries, modules, data models, subsystems, security architecture, design system tokens, workflows, and configuration specifications.

Platform Name	RentPoint	Target Area	Nsukka Urban, Enugu State, Nigeria
Framework	Django 6.1 (Python 3.14 / 3.11+)	Architecture	Modular Multi-App MVT Architecture
Primary Payment Gateway	Paystack (Card, Transfer, USSD)	Platform Fee	8.0% Platform Commission
Database Engine	SQLite (Dev) / PostgreSQL / MySQL	Production Server	Gunicorn + WhiteNoise (Render / Railway)
Email Services	Brevo SMTP / Gmail App Passwords	Document Date	September 2026

Table of Contents

- 1. Executive Overview & System Mission
- 2. Complete Technology Stack & Dependency Matrix (What, Where & Why)
- 3. System Architecture & Directory Layout
- 4. Database Schema & Data Models Specification
- 5. Core Functional Subsystems & Technical Implementation
 - 5.1 Dual-Role Authentication & Password Recovery Subsystem
 - 5.2 Owner Identity & NIN Verification Subsystem
 - 5.3 Listing Management, Dynamic Units & Pricing Simulator
 - 5.4 Booking Lifecycle, Daily Rate Calculation & Real Receipts Engine
 - 5.5 Payment Gateway (Paystack), Escrow & Wallet Settlement Engine
 - 5.6 Site Administration, Moderation & Dispute Resolution
- 6. Frontend Engineering & Custom Design System
- 7. Production Deployment, Environment Configuration & Operational Runbook
- 8. Complete Project File Reference Matrix

1. Executive Overview & System Mission

1.1 Background & Problem Statement

In urban university and commercial towns such as **Nsukka Urban**, access to rental properties, event equipment (canopies, chairs, sound systems), household gadgets, tools, and appliances has historically suffered from several friction points: unverified vendors, lack of price transparency, payment disputes, zero digital audit trail, physical receipts that get lost, and insecurity for equipment owners who rent to strangers.

RentPoint was conceived, architected, and built as a comprehensive, modern, automated web-based rental management platform that connects verified Item Owners with Customers looking to rent items, backed by automated Paystack escrow payments, digital receipts, NIN identity verification, dispute handling, and a digital wallet payout system.

1.2 Target Stakeholders & Roles

The platform enforces distinct access controls and permissions across three principal user categories:

- 1. Customer (Renter):** Searches and filters verified listings in Nsukka Urban, calculates multi-day rental costs with clear formulas, books items, pays via Paystack, receives immediate printable PDF/HTML receipts, tracks active/completed bookings, requests extensions, and files disputes if needed.
- 2. Item Owner:** Creates and manages listings with multiple high-resolution photos, dynamic item unit descriptions (e.g. 50 chairs, 4 pots, 2 sound mixers), daily pricing, views real-time earnings simulations, verifies national identity via NIN and webcam selfie, tracks incoming bookings, approves returns, and withdraws rental earnings to their Nigerian bank account.
- 3. Platform Administrator:** Reviews and approves/rejects Owner NIN verification submissions, reviews transaction disputes, monitors rental volumes, approves bank withdrawal payouts, and manages site-wide configurations.

2. Complete Technology Stack & Dependency Matrix

Every dependency in RentPoint was deliberately selected to ensure zero unnecessary bloat, robust security, zero-cost production capability, and high performance:

Technology / Package	Version	Where Used	Purpose & Technical Rationale
Django	6.1.*	Core Backend (All Apps)	Web framework providing ORM, authentication, CSRF/XSS protection, template engine, CBVs, form validation, and admin.
Python	3.11 - 3.14	Runtime Language	Primary server-side language executing all business logic, financial arithmetic, and security tokens.
Pillow	>=12.3.0	listings/models.py, accounts/models.py	Image processing engine handling listing uploads, thumbnail generation, and NIN verification photo verification.
WhiteNoise	6.*	config/wsgi.py, config/settings.py	Serves compressed static files directly from Gunicorn with cache-busting hashes, eliminating complex Nginx configurations on cloud hosts.
Gunicorn	23.*	Production WSGI Runner	High-concurrency, multi-worker WSGI HTTP server designed for production deployment on Render/Railway/Fly.io.
psycopg2-binary	2.*	config/settings.py	PostgreSQL database adapter enabling connection to free cloud databases (Supabase, Railway, Neon, AWS RDS).
mysqlclient	2.*	config/settings.py	High-performance MySQL driver used when DB_ENGINE=mysql on shared cPanel or cloud MySQL instances.

ReportLab	4.*	docs/, Document Generator	Generates pixel-perfect, secure, standalone PDF documentation and formal system manuals programmatically.
Paystack API	v2 REST / Inline	transactions/views.py, wallet/views.py	Primary Nigerian payment gateway handling Debit Card, Bank Transfer, and USSD payments with HMAC SHA512 webhooks.
Brevo / Gmail SMTP	TLS (Port 587)	accounts/forms.py, config/settings.py	Free SMTP transactional delivery for password recovery links, rental receipts, and account notices.
Vanilla JS	ES6+ Native	static/js/main.js	Zero-dependency client scripting for camera capture, dynamic pricing calculator, gallery zoom, password strength meter.
Vanilla CSS3	Design Tokens	static/css/main.css	Custom design system with CSS custom properties, responsive Flexbox/Grid, print stylesheets, glassmorphism, and animations.

3. System Architecture & Modular Layout

RentPoint follows Django's clean **Separation of Concerns** by organizing features into isolated domain apps. The platform is configured so each app owns its models, forms, views, URLs, tests, and domain templates:

Directory / App	Core Responsibility & Components
config/	Root configuration holding settings.py (environment-driven, auto-switches DBs, email, staticfiles), urls.py (master routing table), and wsgi.py.
accounts/	User identity, RBAC (Customer vs. Item Owner), NIN verification submission, live webcam selfie capture, profile management, and 4-step secure password reset.
listings/	Categories, Item catalog, multi-photo gallery upload, daily pricing, search/filter, condition rating, dynamic quantity units, and owner pricing simulators.
transactions/	Rental checkout, daily rate formula calculation, Paystack payment processing, itemized real receipts, receipt printing, rental status transitions, extensions, and disputes.
wallet/	Financial ledger for Item Owners, escrow balance holds, platform commission calculation (8%), available balance tracking, and bank account withdrawal requests.
core/	Landing page, search discovery, Site Admin dashboard (NIN review, dispute resolution, analytics), custom 404/500 handlers, and site-wide context processors.
static/	Custom design system in css/main.css, client behaviors in js/main.js, and SVG brand assets/favicons in img/.
templates/	Shared global templates: base.html, navigation bars, flash messages, modal popups, error pages, and site admin views.
docs/	Complete operations documentation: RUNBOOK.md, DESIGN.md, email.md, render.md, and rentpointdoc.pdf.

4. Database Schema & Data Models Specification

The database schema enforces referential integrity, decimal precision for currency amounts, indexed query lookups, and lifecycle status machines:

Model Name	Key Fields & Data Types	Business Rules & Relationships
User (accounts)	role (CUSTOMER / ITEM_OWNER) phone_number (CharField 20) verification_status (UNVERIFIED / PENDING / VERIFIED / REJECTED) address (CharField 255)	Extends AbstractUser. Central identity entity for customers, item owners, and site administrators. Enforces role-based views and guards.
OwnerVerification (accounts)	owner (OneToOne → User) full_legal_name (CharField 200) nin (CharField 11) selfie_image (ImageField) nin_front_image (ImageField) nin_back_image (ImageField) status (PENDING / APPROVED / REJECTED)	Collects verified NIN and webcam selfie photos. Overrides save() and delete() to automatically clean up disk image files when updated or deleted.
Category (listings)	name (CharField 100, unique) slug (SlugField 100, unique) description (TextField)	Organizes items (e.g. Household & Kitchen, Sound Systems, Party Equipment, Power & Generators, Construction).

Item (listings)	owner (ForeignKey → User) category (ForeignKey → Category) name (CharField 200) daily_rate (DecimalField 10,2) quantity_available (PositiveIntegerField) condition (EXCELLENT / GOOD / FAIR) is_active (BooleanField)	Represents rental inventory. Computes dynamic units (e.g. '1 spoon', '4 pots', '50 chairs') and checks real-time availability.
Transaction (transactions)	customer (ForeignKey → User) item (ForeignKey → Item) start_date, end_date (DateField) duration_days (IntegerField) quantity (IntegerField) daily_rate (DecimalField) subtotal, platform_fee, total_amount (DecimalField 10,2) status (PENDING / PAID / ACTIVE / COMPLETED / CANCELLED / DISPUTED) payment_reference (CharField 100, unique)	The central contract of a rental. Automatically calculates pricing formula (daily_rate × quantity × days). Manages Paystack payment verification, receipt generation, and escrow release.
Wallet (wallet)	owner (OneToOne → User) balance (DecimalField 12,2) pending_balance (DecimalField 12,2)	Maintains ledger balance for each Item Owner. Escrow holds funds during active rentals and credits available balance upon verified return.
WithdrawalRequest (wallet)	wallet (ForeignKey → Wallet) amount (DecimalField 10,2) bank_name, account_number, account_name status (PENDING / APPROVED / REJECTED)	Enables item owners to withdraw earned rental income. Enforces minimum withdrawal threshold (₦1,000) and balance checks.

5. Core Functional Subsystems & Technical Implementation

5.1 Dual-Role Authentication & Password Recovery Subsystem

RentPoint implements strict role-based registration via distinct entry points (/accounts/signup/customer/ and /accounts/signup/item-owner/) to prevent accidental role ambiguity. Password authentication uses Django's Argon2/PBKDF2 SHA256 password hashers.

Password Reset Engineering:

- 1. Timing Attack & Enumeration Protection:** Submitting an unknown email returns the identical success confirmation screen, preventing malicious actors from harvesting registered emails.
- 2. Branded HTML Email Dispatch:** Sends responsive, cross-client HTML emails (with dark navy header, emerald CTA button, and 24-hour expiration token) alongside a plain-text fallback.
- 3. Interactive Client-Side Strength Meter:** Built in static/js/main.js with dynamic 4-bar colored feedback (Weak → Fair → Good → Strong) and real-time rule checklist.
- 4. One-Time Token Invalidation:** Once used or expired, tokens immediately invalidate, redirecting to an explicit 'Link Expired or Invalid' recovery screen.

5.2 Owner Identity & NIN Verification Subsystem

To build customer trust, Item Owners must verify their identity. The owner submits their 11-digit NIN, live front and back card images, and a live webcam photo captured directly via HTML5 navigator.mediaDevices.getUserMedia(). Submissions enter a PENDING queue reviewed in the Site Admin panel.

5.3 Booking Lifecycle, Daily Rate Calculation & Real Receipts Engine

Rentals are calculated per day rather than ambiguous event units. The system applies a rigorous pricing formula:

Daily Calculation Formula: $\text{Total} = \text{Daily Rate} \times \text{Quantity} \times \text{Duration (Days)}$

Upon payment, RentPoint generates a **Real Itemized Receipt** (_receipt_card.html) structured like modern financial enterprise receipts (Stripe/Square/Apple), featuring:

- **Clean Proportions:** 44% Item Description, 18% Daily Rate, 10% Qty count, 12% Duration, 16% Amount.
- **Structured Metadata Badges:** Folder category pill tag and green verified condition badge.
- **Mathematical Breakdown Badge:** Explicit rate verification showing Calculation: $\blacksquare \text{Rate} \times \text{Qty} \times \text{Days} = \blacksquare \text{Total}$.
- **High-DPI Print Stylesheet:** Dedicated @media print CSS ensuring receipts fit standard A4/invoice paper with headers, borders, and barcodes without browser clutter.

5.4 Payment Gateway (Paystack), Escrow & Wallet Settlement Engine

Payments are processed through Paystack Inline/Standard. When a customer pays, funds are not sent directly to the owner. Instead, RentPoint acts as an **Escrow Agent**:

1. Customer pays Total Amount → Webhook verifies HMAC SHA512 signature → Transaction status becomes PAID.
2. Net amount (Total - 8% Commission) is recorded into the Owner's Wallet as pending_balance.
3. When the rental period ends and the item is returned in good condition, the status transitions to COMPLETED and the funds automatically move from pending_balance to balance.
4. The Item Owner can initiate a bank withdrawal request to any verified Nigerian bank account.

6. Frontend Engineering & Custom Design System

RentPoint's frontend is entirely dependency-free: no heavy npm build steps, Webpack configurations, or bloated CSS frameworks. It uses vanilla CSS3 custom properties and clean vanilla JavaScript for lightning-fast page loads.

Design Token / Variable	Value / Hex	Application & Usage
--color-primary-navy	#0A2342	Primary brand identity, page headers, navigation bar, authority badges, and button active states.
--color-accent-teal	#00A896	Primary CTA gradients, interactive links, success highlights, active tabs, and focus outlines.
--color-accent-amber	#D97706	Item owner badges, warnings, pending statuses, and review stars.
--color-bg	#F8FAFC	Clean modern background tone preventing eye strain and enhancing contrast.
--font-display	'Outfit', sans-serif	Typography for high-impact headlines, numbers, currency values, and card titles.
--font-body	'Plus Jakarta Sans'	Clean, readable body copy, form inputs, metadata tags, and table cells.
--font-mono	'JetBrains Mono'	Financial receipts, reference codes, timestamps, and currency calculations.

7. Production Deployment & Operations Runbook

RentPoint is architected to run seamlessly on both zero-config local development environments and multi-container cloud infrastructure (Render, Railway, Fly.io, AWS, Heroku):

7.1 Automated Database Switching:

Setting `DB_ENGINE=postgres` and providing a `DATABASE_URL` (e.g. from Supabase) automatically configures PostgreSQL with SSL enforcement. In local development, leaving it unset defaults safely to SQLite.

7.2 Static Asset Delivery:

When `DJANGO_DEBUG=False`, `WhiteNoise` automatically compresses static files into Gzip/Brotli format and applies permanent cache headers (Cache-Control: max-age=31536000).

7.3 Production Security Hardening:

- **SSL / HTTPS Enforcement:** `SECURE_SSL_REDIRECT = True` and `SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https')`.
- **HSTS Preload:** `SECURE_HSTS_SECONDS = 31536000` (1 year) with subdomains enabled.
- **Secure Cookies:** `SESSION_COOKIE_SECURE = True` and `CSRF_COOKIE_SECURE = True`.

8. Complete Project File Reference Matrix

File Path	Type	Detailed Functionality & Role
-----------	------	-------------------------------

manage.py	CLI Entry	Django's command-line runner for migrations, tests, local dev server, and custom management commands.
config/settings.py	Settings	Master configuration: database engine switching, email backend, WhiteNoise, Paystack keys, and security hardening.
config/urls.py	Root Router	Top-level URL dispatch table mounting accounts, listings, transactions, wallet, core, and media handlers.
accounts/models.py	Models	Defines User (multi-role) and OwnerVerification with automated media deletion.
accounts/forms.py	Forms	Defines SignUpForm, LoginForm, RentPointPasswordResetForm, and OwnerVerificationForm.
accounts/views.py	Views	Implements customer/owner registration, login/logout, password reset CBVs, and identity verification.
listings/models.py	Models	Defines Category, Item (daily rate, quantity, condition), and ItemImage.
listings/views.py	Views	Listing catalog, search/filter, detail view, creation/edit forms, and photo management.
transactions/models.py	Models	Defines Transaction, RentalExtension, Dispute, and Review models.
transactions/views.py	Views	Checkout handler, Paystack payment webhook, real itemized receipt rendering, return confirmation.
wallet/models.py	Models	Defines Wallet, WalletTransaction, and WithdrawalRequest.
wallet/views.py	Views	Earnings overview, ledger statement, and bank withdrawal request submission.
core/views.py	Views	Landing page, custom Site Admin dashboard (NIN reviews, disputes, withdrawals, analytics), 404/500 handlers.
static/css/main.css	CSS System	Comprehensive design system tokens, typography, cards, receipt styling, and print stylesheets.
static/js/main.js	JavaScript	Camera capture, image zoom gallery, dynamic pricing calculator, and password strength meter.
docs/email.md	Docs	Step-by-step setup guide for 100% free forever transactional email delivery (Gmail App Password & Brevo).
docs/RUNBOOK.md	Docs	Production deployment manual, environment variable reference, database commands, and maintenance.